1. Introduction

The sinking of the Titanic is perhaps the most infamous sinking in maritime history. Out of the estimated 2242 people onboard 1635 perished. One of the reasons Titanic's fate is interesting is that whether a person survived or not had so much to do with their background. For example, wealthy passengers were much more likely to survive compared to others. There are multiple other factors to take into account, and that is also one of the goals of this paper, to find out which factors had the most impact on the chance of survival.

The paper consists of the following:

- Section 2 specifies the problem and explains more about the dataset being used.
- Section 3 focuses on machine learning methods used in this project.
- Section 4 explains the results which were gotten from the machine learning algorithms.
- Section 5 tells the conclusion, whether the goal was achieved or not.
- Section 6 has a list of all the references used in this project.
- Section 7 consists of all graphs and tables referred in the text.

2. Problem formulation

The goal of this machine learning project is to predict whether a passenger survived the Titanic disaster, and which factors had the most impact in their chances of survival. The training dataset has 891 passengers and is sourced from Kaggle. Each data point includes various features about the passenger, such as:

- Name
- Sex
- Age
- Cabin (if known)
- Passenger class (Pclass)
- Number of siblings/spouses aboard (SibSp)
- Number of parents/children aboard (Parch)
- Ticket number
- Fare
- Embarkation port (Embarked)
- Survival status (0 if did not survive and 1 if survived)

The label to predict is the Survival status. A separate test dataset with 417 passengers is also sourced from Kaggle, which includes all features except the survival label.

3. Methods

Preprocessing:

The dataset contains 891 data points. The dataset is mostly clean, but some of the features need preprocessing. These features are “Cabin”, “Embarked” and “Age”. Some of the features are not needed for training, these are “Name” and “Ticket number”. These features do not affect the survival status of the passenger, and therefore they are excluded.
The features selected for training are:

- Sex
- Age
- Cabin
- Pclass
- SibSp
- Parch

Multiple passengers had “Cabin” feature missing, and these missing values were marked as “U” for unknown. The cabin number itself is not important, the deck is. Therefore I extracted the deck identificator (letters A to G) from the cabin feature, and encoded them to numbers 1 to 7, 7 being the highest passenger deck (A) and 1 being the lowest (G). Unknown cabins were encoded to 0.

Other features that had missing values were “Embarked” and “Age”. For the embarkation port the missing values were changed to the median embarkation port of passengers. The same was done to passenger ages, missing ages were changed to the median age of all passengers.

Models:

The first model I chose was logistic regression because it is pretty simple and effective model for binary tasks like predicting survival on the Titanic. It provides a clear mathematical relationship between the input features and the probability of survival, making the model easy to understand. This is especially useful when we want to understand which factors most influence the outcome. Logistic regression is also fast to train, works well with a small dataset like the titanic dataset, and serves as a strong base for comparison with other complex models. Logistic regression uses logistic loss as the loss function, which is good for binary classification.

The second chosen model is Random forest, which is a good option for this problem because it can capture nonlinear relationships between features. For example, the model can learn that survival probability may depend on a combination of being of a certain age and in first class, or having a cabin on a specific deck. Random Forest uses gini impurity or entropy to evaluate the quality of splits when building trees. These criteria are effective for classification tasks and require no extra loss function. The parameters for Random Forest were n_estimators=500, max_depth=7, random_state=42.

Validation:

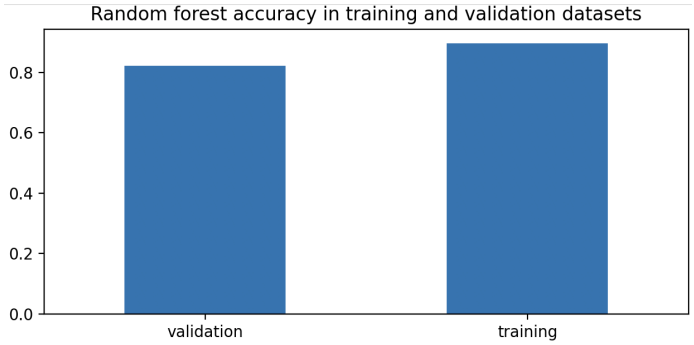
The dataset was split into training and validation sets with 80/20 split. Kaggle provides a separate testing dataset, but it doesn't have the survival status feature, so it can't be used for validation. The 80/20 split was a good balance so that there were enough data points for the training set but also enough for validation.

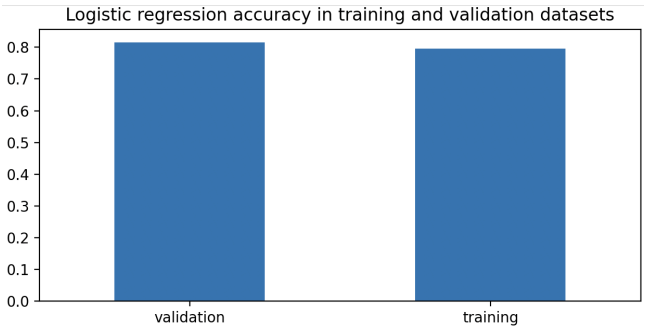
4. Results

Model comparison

Two different machine learning models were used in this study, Random Forest and Logistic Regression. Random Forest achieved a training accuracy of 89,7% and validation accuracy of 82,1%. For logistic regression the corresponding values were 79,6% for training and 81,6% for validation. In other words, random forest is better at generalization, which can be seen from the smaller gap between training and validation accuracy. However Logistic regression is almost as good in validation accuracy.

When looking at the detailed classification reports (found in appendices) we can see that both models give fairly balanced scores for precision, recall and f1-scores. However Random Forest has slightly better performance in these scores.





Model choice

From the two models Random Forest is overall the better performing machine learning model. Random forest has higher accuracy in validation and training data. Random Forest has also the ability to see nonlinear relationships between different features, and this is especially useful in datasets like the titanic dataset.

Test error

The test error of random forest is 1 - validation accuracy, so approximately 17,9%. This means that the model correctly predicts about 82,1% validation passenger list.

5. Conclusion

I had multiple main goals in this project. I wanted to find out if machine learning models can correctly determine which passengers survived, and what were the most impactful factors. The result was that the model was correct on approximately 82% of the passengers. This shows that machine learning models can predict the surviving passengers, but the model is not perfect.

The biggest issue appeared when testing the model on made-up passengers. It was often too pessimistic, especially for males. This likely reflects the dataset's bias since most male passengers didn't survive. Also, the dataset is incomplete and missing some passenger details, which, along with preprocessing, may have affected the model's accuracy.

The most impactful factors for survival according to the model were (graph in appendices):

feature	Class	Sex	Age	SibSp	Parch	Fare	Port	Deck
impact	9,95%	37,43%	13,89%	5,06%	4,02%	17,49%	3,38%	8,77%

As we can see, the values the model gives about different factors are fairly accurate. Sex of the passenger along with age and wealth had the most impact on survival on the real sinking. However, fare and class might overlap, because the ticket price depended on the passenger class. Also, the deck factor is not that accurate, because most of the passengers in the dataset did not have that information.

For further improvement I think more complex models would be the best way to make the result better. Also techniques like cross-validation to estimate performance better is a good idea. Overall I think that I achieved the project goals, and the model was good enough at estimating passenger survival status.

6. References

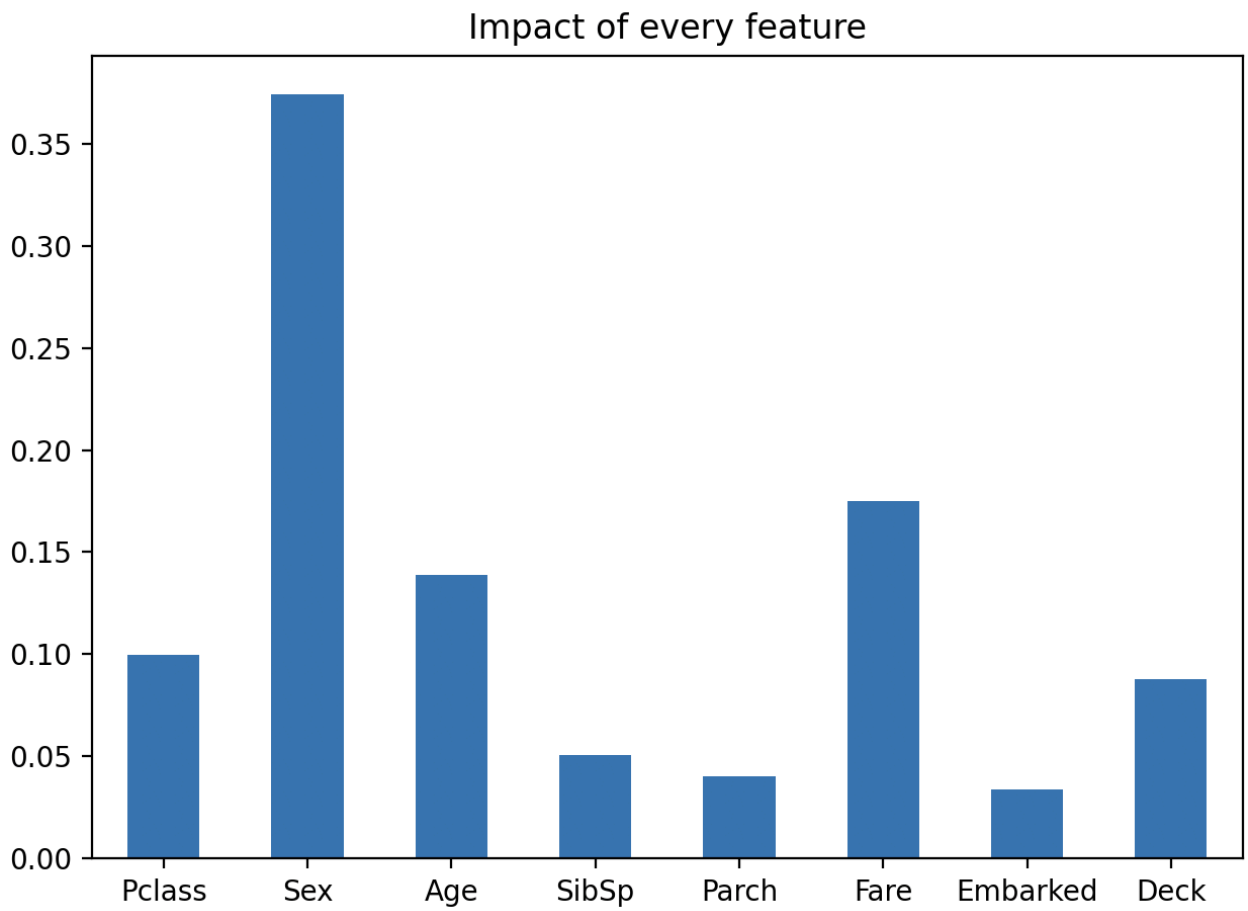
Titaic - Machine Learning from Disaster, Available from Kaggle: https://www.kaggle.com/competitions/titanic

Pandas documentation, available from pandas website: https://pandas.pydata.org/docs/

ChatGPT was used in this project for research. All text and code was written by me.

7. Appendices

Impact of every single feature on the chance of survival



Classification reports, on the leftmost column 0 is for not survived, 1 is for survived.

```
----------------------------------
Random Forest
----------------------------------
Val Accuracy: 0.8212290502793296
Train accuracy: 0.8974719101123596
              precision    recall  f1-score   support

           0       0.82      0.89      0.85       105
           1       0.82      0.73      0.77        74

    accuracy                           0.82       179
   macro avg       0.82      0.81      0.81       179
weighted avg       0.82      0.82      0.82       179
```

```
-----------------------------------
Logistic regression
-----------------------------------
Val Accuracy: 0.8156424581005587
train accuracy: 0.7963483146067416
              precision    recall  f1-score   support

           0       0.83      0.86      0.85       105
           1       0.79      0.76      0.77        74

    accuracy                           0.82       179
   macro avg       0.81      0.81      0.81       179
weighted avg       0.81      0.82      0.82       179
```

ready

October 7, 2025

[ ]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report
import numpy as np

pd.set_option('future.no_silent_downcasting', True)

# The training csv
df = pd.read_csv('train.csv')

# Giving missing info values. Missing cabins get U for Unknown, missing␣
↪embarkation ports the most common, and missing Age is set to median.
df['Cabin'] = df['Cabin'].fillna('U')
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
df['Age'] = df['Age'].fillna(df['Age'].median())


# Making new column "Deck" for the dataset, because the cabin number itself is␣
↪not that important, the deck is
df['Deck'] = df['Cabin'].str[0]

# Removing unecessary columns
df = df.drop(columns=['Cabin', 'Ticket', 'Name', 'PassengerId'])

# Encoding decks (A to G) to numbers, with A being the highest deck and␣
↪therefore getting the highest value (7). Unknown deck gets 0. T is a single␣
↪error, and that is converted to 0
df['Deck'] = df['Deck'].replace('A', 7)
df['Deck'] = df['Deck'].replace('B', 6)
df['Deck'] = df['Deck'].replace('C', 5)
df['Deck'] = df['Deck'].replace('D', 4)
df['Deck'] = df['Deck'].replace('E', 3)
df['Deck'] = df['Deck'].replace('F', 2)
df['Deck'] = df['Deck'].replace('G', 1)
```

```python
df['Deck'] = df['Deck'].replace('U', 0)
df['Deck'] = df['Deck'].replace('T', 0)



# encoding embark port
df['Embarked'] = df['Embarked'].replace('C', 1)
df['Embarked'] = df['Embarked'].replace('Q', 2)
df['Embarked'] = df['Embarked'].replace('S', 3)


# Encoding sex
df['Sex'] = df['Sex'].map({'male': 1, 'female': 0})


df['Fare'] = df['Fare'].apply(lambda x: np.log(x + 1))

# Split the csv to train and testing
X = df.drop(columns=['Survived'])
y = df['Survived']


X_train, X_val, y_train, y_val = train_test_split(X, y, test_size = 0.2,␣
 ↪random_state = 42)


# training Random forest
model = RandomForestClassifier(n_estimators=500, max_depth=7, random_state=42)
model.fit(X_train, y_train)

print("")
print("----------------------------------")
print("Random Forest")
print("----------------------------------")

# TEST
y_pred = model.predict(X_val)
#validation set accuracy
print("Val Accuracy:", accuracy_score(y_val, y_pred))
#training accuracy
y_pred_train = model.predict(X_train)
print("Train accuracy:", accuracy_score(y_train, y_pred_train))

print(classification_report(y_val, y_pred))
print(model.feature_importances_)

print(df.groupby(['Pclass', 'Sex'])['Survived'].mean())
```

```python
#space
print("")
print("***")
print("")


print("----------------------------------")
print("Logistic regression")
print("----------------------------------")

#Logical regression
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train_log, X_val_log, y_train_log, y_val_log = train_test_split(X_scaled, y,␣
 ↪test_size = 0.2, random_state = 42)

model_log = LogisticRegression()
model_log.fit(X_train_log, y_train_log)

y_pred_log = model_log.predict(X_val_log)

#val accuracy
print("Val Accuracy:", accuracy_score(y_val_log, y_pred_log))
#train accuracy
y_pred_train_log = model_log.predict(X_train_log)
print("train accuracy:", accuracy_score(y_train_log, y_pred_train_log))
print(classification_report(y_val_log, y_pred_log))



print("********************")

#Test passenger, used for testing the model with single data point
test_passenger = pd.DataFrame([{
    'Pclass': int(input("class: ")),
    'Sex': int(input("Sex: ")),
    'Age': float(input("Age: ")),
    'SibSp': int(input("Number siblings/spouces: ")),
    'Parch': int(input("Number of children/parents: ")),
    'Fare': float(input("Ticket fare: ")),
    'Embarked': int(input("Embark port: ")),
    'Deck': int(input("Deck: "))

}])
```

```python
forest_test = model.predict(test_passenger)
forest_prob = model.predict_proba(test_passenger)

test_scaler = StandardScaler()
test_passenger = test_scaler.fit_transform(test_passenger)

log_prob = model_log.predict_proba(test_passenger)
log_test = model_log.predict(test_passenger)


print("Random forest")
print("Survived" if forest_test[0] == 1 else "Did not survive")
print("Probability: ", forest_prob[:, 1])

print("***")

print("Logistic Regression")
print("Survived" if log_test[0] == 1 else "Did not survive")
print("Probability: ", log_prob[:, 1])
```